

## Lab Test 2

**[25 marks]****Coding time: 1 hour and 30 mins****Instructions on how to download the resource files:**

1. Before the test begins, you will be instructed to download the resource files from eLearn LMS.

**General Instructions:**

1. You will take the lab test on your personal laptop. Remember to disable all AI-based extensions/tools in your IDE if any.
2. You are not allowed to communicate with anyone or access any web resources (except for downloading/uploading files from/to eLearn) during the test. **Anyone caught doing so during the test WILL be considered as cheating and subjected to disciplinary actions.**
3. You may refer to any file on your laptop during the test.
4. Make sure your code can generate the same output as we show in the sample runs. You may be penalized for missing spaces, missing punctuation marks, misspellings, etc. in the output.
5. Do not hardcode. We will use different test cases to test and grade your solutions.
6. Python script files that cannot be executed will NOT be marked and hence you will be awarded 0 marks. You may wish to comment out the parts in your code which cause execution errors.
7. Include your name as author in the comments of all your submitted source files. For example, include the following block of comments at the beginning of each source file you need to submit.

```
# Name      : BAI She Jing  
# Email ID: shejing.bai.2023
```

**Instructions on how to submit your solutions:**

1. When the test ends, zip up all the files required for submission in a zip archive. The name of the zip archive should be your email ID. For example, if your email is `shejing.bai.2023@scis.smu.edu.sg`, you should name the archive as **shejing.bai.2023.zip**. You may be penalized for not following our instructions.
2. Once everybody has zipped up his/her files, your invigilator will instruct you to submit your solutions as a single zip file to eLearn under Assignments.

**Question 1****[ 9 marks ]**

When registering a car in Singapore, you are required to buy a Certificate of Entitlement (COE) and pay additional duties. Fortunately, if you deregister your car before it reaches the 10-year mark, you can receive a partial refund of these expenses. The deregistration value is composed of two components:

- the Certificate of Entitlement (COE) rebate;
- the Preferential Additional Registration Fee (PARF) rebate.

In short,

**Deregistration value = COE rebate + PARF rebate**

Calculating the COE rebate:

The COE you purchase allows you to drive your car for a 10-year duration. You have the option to recover the unused portion of the COE when you deregister your vehicle. The COE rebate is rounded down to the nearest dollar.

**COE rebate = COE paid x Number of days left / (10 Years x 365 Days)**

For example, if John paid a COE of **\$49,430** upon registration and decides to deregister the car after owning it for a total of **578 days (3072 days left)**, his COE rebate would be:

$$\text{COE rebate} = 49430 \times 3072 / (10 \times 365) = \$41602$$

Calculating the PARF rebate:

The Additional Registration Fee (ARF) is a fee paid during car registration. Depending on the duration of your car ownership, you can qualify to receive a partial refund of this fee. The PARF rebate is rounded down to the nearest dollar.

**PARF rebate = ARF x PARF rebate percentage**

| Number of years owned                    | PARF rebate percentage |
|------------------------------------------|------------------------|
| Not exceeding 5 years                    | 75%                    |
| Above 5 years but not exceeding 6 years  | 70%                    |
| Above 6 years but not exceeding 7 years  | 65%                    |
| Above 7 years but not exceeding 8 years  | 60%                    |
| Above 8 years but not exceeding 9 years  | 55%                    |
| Above 9 years but not exceeding 10 years | 50%                    |

Note: "Above X years but not exceeding Y years" means number of years owned is strictly greater than X years but lesser or equal to Y years.

For example, if John paid an ARF of **\$9,370** upon registration and decides to deregister the car after owning it for a total of **578 days**, his PARF rebate would be:

$$\text{PARF rebate} = 9370 \times 0.75 = \$7027$$

Notes for Q1:

- Assume all cars do not exceed 10 years old. You can assume that 1 year = 365 days.
- Number of days left is calculated by subtracting the days owned from the 10-year duration.

- You can assume that all arguments given to the below functions are valid.

### Part A (Difficulty Level: \*)

Implement a function called `compute_coe_rebate` in `q1a.py`, the function takes in 1 parameter:

- `car_info` (type: tuple): A non-empty tuple that describes the attributes of the car as follow:
  - car name (type: str),
  - number of days owned (type: int),
  - a non-empty list of 2 items consisting of the *COE paid* (type: str), *ARF paid* (type: str) for the car respectively (type: list). Note that the COE paid and ARF paid are always in int values.

The function **returns the COE rebate rounded down** to the nearest dollar (type: int).

- Hint: You can use the `int()` function to round down a float value.

Examples:

- `car_1 = ("Peugeot 308 SW 1.2A PureTech EAT6 Allure", 578, ["$49430", "$9370"])`  
`compute_coe_rebate(car_1)` returns 41602
- `car_2 = ("Volkswagen Jetta 1.4A TSI", 3074, ["$28389", "$20546"])`  
`compute_coe_rebate(car_2)` returns 4480
- `car_3 = ("Mercedes-Benz SLK-Class SLK200K", 3403, ["$5982", "$50320"])`  
`compute_coe_rebate(car_3)` returns 404
- `car_4 = ("BMW Z4 sDrive35i", 3285, ["$19833", "$71269"])`  
`compute_coe_rebate(car_4)` returns 1983

Use `q1a_test.py` to test your code. Your `q1a.py` should contain the function definition of `compute_coe_rebate`. We will only grade your `q1a.py` using a different test script.

### Part B (Difficulty Level: \*/\*\*)

Implement a function called `compute_parf_rebate` in `q1b.py`, the function takes in 1 parameter:

- `car_info` (type: tuple): This input is as defined in Part A.

The function **returns the PARF rebate rounded down** to the nearest dollar (type: int).

- Hint: You can use the `int()` function to round down a float value.

Using the same examples of `car_info` in Part A:

- `compute_parf_rebate(car_1)` returns 7027
- `compute_parf_rebate(car_2)` returns 11300
- `compute_parf_rebate(car_3)` returns 25160

- `compute_parf_rebate(car_4)` returns 39197

Use `q1b_test.py` to test your code. Your `q1b.py` should contain the function definition of `compute_parf_rebate`. We will only grade your `q1b.py` using a different test script.

### Part C (Difficulty Level: \*\*)

In addition to the option of deregistering the car, a vehicle owner could choose to sell their car on the second-hand market. The owner of a used car platform has tasked you with creating a program to facilitate buyers in discovering the vehicle with the highest deregistration value.

Implement a function called `get_highest_value` in `q1c.py`, the function takes in 1 parameter:

- `car_info_list` (type: list): A list of car information, describing the attributes of the cars available on the second-hand car marketplace.
  - Note: this is a list of `car_info` (type: tuple) as defined in Part A.

The function returns the name of the car (type: str) with the highest deregistration value among the candidates. If two cars have the same deregistration value, return the first car in the list.

Notes:

- You should import and use the function `compute_coe_rebate` from `q1a.py`.
- You should import and use the function `compute_parf_rebate` from `q1b.py`.
- We will test Part C with the correct implementations of Part A and Part B.

Examples:

```
car_info_list = [('BMW 7 Series 730Li Sunroof', 2487, ['$77340', '$73579']),
('Peugeot 308 SW 1.2A PureTech EAT6 Allure', 578, ['$49430', '$9370']),
('Audi A7 Sportback 2.8A FSI Quattro', 2671, ['$56889', '$62923']),
('Mercedes-Benz SLK-Class SLK200K', 3403, ['$5982', '$50320']), ('Audi A4
1.8A TFSI MU', 1978, ['$62000', '$32605']), ('Volkswagen Jetta 1.4A TSI',
3074, ['$28389', '$20546']), ('BMW Z4 sDrive35i', 3285, ['$19830',
'$71269']), ('Audi A4 1.8A TFSI V2', 1979, ['$62038', '$32605']),
('Volkswagen Golf 1.4A TSI', 1409, ['$64900', '$5000'])]
```

- `get_highest_value(car_info_list)` returns 'BMW 7 Series 730Li Sunroof'
- `get_highest_value(car_info_list[2:])` returns 'Audi A7 Sportback 2.8A FSI Quattro'
- `get_highest_value(car_info_list[4:])` returns 'Audi A4 1.8A TFSI MU'

Use `q1c_test.py` to test your code. Your `q1c.py` should contain the function definition of `get_highest_value`. We will only grade your `q1c.py` using a different test script.

## Question 2

**[ 7 marks ]**

### Part A (Difficulty Level: \*)

Implement a function called `get_special_word_count` in `q2a.py`. This function takes in 1 parameter which is the name of a text file called `filename` (type: `str`).

We assume that the text file `filename` has at least one line, and words on a line are separated by a single space character only, i.e., " ". For example, `sample1.txt` has 2 lines with a total of 15 words, as shown below.

```
Wow! There are 3 lazy Dogs!
Hey, how much was this? It's just $10 dollars.
```

The function returns the number of words containing a special character in the file. We assume that each word in the file has at most a single special character, which can be one of the following: `. , ; : ! ? ( ) [ ] { } ~ & * \ / + - = % $ # @ ^ _ ' " < >`

- Hint: a string `special_chars` containing all the special characters that you need to consider has been included in the starting code. In your code, for each character `ch` in a word, you can check if `ch` is in `special_chars`.

Examples:

- `get_special_word_count ("sample1.txt")` returns 7 because the file has seven words containing a special character: "Wow!", "Dogs!", "Hey, ", "this?", "It's", "\$10", and "dollars."

Use `q2a test.py` to test your code. Your `q2a.py` should contain the function definition of `get_special_word_count`. We will only grade your `q2a.py` using a different test script.

### Part B (Difficulty Level: \*\*)

Implement a function called `generate_loan_report` in `q2b.py`. This function takes in two parameters called `filename` (type: `str`) and `bank_code` (type: `str`).

We assume that the text file `filename` has at least one line. For example, `loandata.txt` has multiple lines, as shown below.

```
S**11111,DBS000001,HOM-LOAN,$100000
S**11111,SCB000001,BIZ-LOAN,$100000
S**11111,UOB000001,CAR-LOAN,$125000
T**11112,DBS000002,EDU-LOAN,$125000
S**11115,SCB000002,CAR-LOAN,$100000
```

Each line in the file contains the following comma-separated values in this order: NRIC (type: `str`), Branch ID (type: `str`), Loan Type (type: `str`), and Loan Amount (type: `str`). The first three characters of the Branch ID represent the `bank_code`. Note that it is perfectly fine for two different lines to have the same text. All letters in the file's content are in capital. The Loan Amount is always an `int` after stripping away the \$ character.

For example, in the line `S**11111,UOB000001,CAR-LOAN,$125000`:

`S**11111` denotes the customer NRIC, `UOB000001` denotes the Branch ID, `CAR-LOAN` denotes the loan type, and `$125000` denotes the loan amount of 125000. The first three characters of the Branch ID, i.e., `UOB`, denote the `bank_code`.

The function `generate_loan_report` will return a list of tuples of loan information for the given `bank_code` and the total loan amount `total_amount` from all the branches having this `bank_code`. The order of the tuples in the list follows the order of the corresponding lines in the file. The total loan amount tuple must be in this form `('Total Loan Amount', total_amount)`. It is the last tuple in the returned list if the list is not empty.

More examples:

- `generate_loan_report("loandata.txt", "DBS")` returns  
`[('S**11111', 'DBS000001', 'HOM-LOAN', 100000), ('T**11112', 'DBS000002', 'EDU-LOAN', 125000), ('Total Loan Amount', 225000)]`
- `generate_loan_report("loandata.txt", "UOB")` returns  
`[('S**11111', 'UOB000001', 'CAR-LOAN', 125000), ('Total Loan Amount', 125000)]`
- `generate_loan_report("loandata.txt", "SCB")` returns  
`[('S**11111', 'SCB000001', 'BIZ-LOAN', 100000), ('S**11115', 'SCB000002', 'CAR-LOAN', 100000), ('Total Loan Amount', 200000)]`
- `generate_loan_report("loandata.txt", "ABC")` returns `[]`

Use `q2b_test.py` to test your code. Your `q2b.py` should contain the function definition of `generate_loan_report`. We will only grade your `q2b.py` using a different test script.

### Question 3

[ 5 marks ]

After 26 years, Ash Ketchum is retiring as the greatest Pokémon trainer. He now has ample time to write his memoirs and organize the Pokédex, the database of all the Pokémon he encountered during his adventures.<sup>1</sup>

#### Part A (Difficulty Level \*\*/\*\*\*)

Implement a function called `load_pokedex` in `q3a.py`.

This function accepts:

- A filename of a text file (`str:filename`). See Figure 1.

This function returns:

- A list of dictionaries. See Figure 2.
  - Each dictionary contains the record of one Pokémon.
  - Each dictionary entry is one attribute for that Pokémon.

#### Text File Format

<sup>1</sup> Rotom Dex is no longer employed by Ash. It is pursuing a career selling NFTs and other crypto-assets to unsophisticated investors.

There are two text files in your code distribution. For both parts of this question, the testing script will first exercise your code with `pokedex_abridged.txt`, a reduced dataset. If your code passes the initial tests, it will test with `pokedex.txt`, which contains more detailed data on additional Pokémon.<sup>2</sup>

- All columns are delimited with commas. There are no commas in the data itself.
- The first line in the file is the header. It gives a name to each column (field) in the dataset.
  - See Figure 3 for a hint on how to process the header.
- Each subsequent line is a record for a different Pokémon.

Figure 1 shows the contents of `pokedex_abridged.txt`. In this example, there are four fields for each record: `pokedex_number(int)`, `name(str)`, `type_2(str)`, and `height_m(float)`. Other files will have a different number of fields with different field names and data types.

```
pokedex_number,name,type_2,height_m
1,Bulbasaur,Poison,0.7
2,Ivysaur,Poison,1.1
3,Venusaur,Poison,2.1
3,Mega Venusaur,Poison,2.4
4,Charmander,,0.6
5,Charmeleon,,1.1
6,Charizard,Flying,1.7
6,Mega Charizard X,Dragon,1.7
6,Mega Charizard Y,Flying,1.7
7,Squirtle,,0.5
8,Wartortle,,1.1
```

*Figure 1: Contents of pokedex\_abridged.txt*

The values should be stored as one of **four** data types.

- `None` Empty values signify the absence of data. Store them as `NoneType`.  
There is an empty field in the last line of `pokedex_abridged.txt` in Figure 1
- `int` If all characters are digits.
- `float` If all characters are digits except for a single decimal point.
- `str` For all other cases.

If the `load_pokedex` function takes the filename `pokedex_abridged.txt` as an input parameter, it will return the list of dictionaries in Figure 2. Each row in the input file corresponds to a dictionary in the output list. Fields that contain numeric data should have the appropriate type, either `int` or `float`. Empty fields in the input file are stored as `NoneType` in the dictionaries.

```
[{'pokedex_number': 1, 'name': 'Bulbasaur', 'type_2': 'Poison', 'height_m': 0.7},
 {'pokedex_number': 2, 'name': 'Ivysaur', 'type_2': 'Poison', 'height_m': 1.1},
 {'pokedex_number': 3, 'name': 'Venusaur', 'type_2': 'Poison', 'height_m': 2.1},
 {'pokedex_number': 3, 'name': 'Mega Venusaur', 'type_2': 'Poison', 'height_m': 2.4},
 {'pokedex_number': 4, 'name': 'Charmander', 'type_2': None, 'height_m': 0.6},
 {'pokedex_number': 5, 'name': 'Charmeleon', 'type_2': None, 'height_m': 1.1},
 {'pokedex_number': 6, 'name': 'Charizard', 'type_2': 'Flying', 'height_m': 1.7},
 {'pokedex_number': 6, 'name': 'Mega Charizard X', 'type_2': 'Dragon', 'height_m': 1.7},
 {'pokedex_number': 6, 'name': 'Mega Charizard Y', 'type_2': 'Flying', 'height_m': 1.7},
 {'pokedex_number': 7, 'name': 'Squirtle', 'type_2': None, 'height_m': 0.5},
 {'pokedex_number': 8, 'name': 'Wartortle', 'type_2': None, 'height_m': 1.1}]
```

*Figure 2: Output of load\_pokedex corresponding to pokedex\_abridged.txt in Figure 1.*

Notes:

<sup>2</sup> This dataset based on: <https://www.kaggle.com/datasets/mariotormo/complete-pokemon-dataset-updated-090420>

- **Do not hardcode any field names or assume the number of fields in the data file.** We will test your code with a different data file which could have a different number of fields with different field names and data types.
- The order of the dictionaries in the list should be the same as the lines in the data file.
- The order of the fields in the dictionaries does not matter.
- You can use the method in Figure 3 to extract the header.

```
with open(filename) as fp:
    # extract the header
    header = fp.readline()
    # loop through the remaining lines
    for line in fp:
        # more code here
```

Figure 3: Use `readline()` to extract the header from the first line of the file.

Use `q3a_test.py` to test your code. Your `q3a.py` should contain the function definition of `load_pokedex`. We will only evaluate your `q3a.py` with a different test script.

## Part B (Difficulty Level \*\*/\*\*\*)

Ash may want to know only a Pokémon's name and height and not their type or Pokédex number. Ash created PQL (Pokédex Query Language) to extract only the requested fields from the database.

A PQL query is a string that always starts with the keyword `select` followed by zero or more field names. They are separated by a single space character.

Create a function `select_fields` in `q3b.py`.

This function accepts:

- The database (`list: pokedex`). This is the list of dictionaries returned by Part A of this question. The keys and values in each dictionary will change, depending on the content of the data file used in Part A.
- A PQL query (`str: query`)

This function returns:

- A list of dictionaries containing only the selected fields.

Some examples using the dataset from `pokedex_abridged.txt`:

|                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Query:</b> <code>select name</code>                                                                                                                                                                                                                                                                 |
| <code>[{'name': 'Bulbasaur'}, {'name': 'Ivysaur'}, {'name': 'Venusaur'},<br/>{'name': 'Mega Venusaur'}, {'name': 'Charmander'}, {'name': 'Charmeleon'},<br/>{'name': 'Charizard'}, {'name': 'Mega Charizard X'}, {'name': 'Mega Charizard Y'},<br/>{'name': 'Squirtle'}, {'name': 'Wartortle'}]</code> |

Figure 4: This query returns a list of dictionaries with the `name` field, for each dictionary in the `pokedex` parameter.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Query:</b> <code>select name height_m weight kg</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>[{'name': 'Bulbasaur', 'height_m': 0.7}, {'name': 'Ivysaur', 'height_m': 1.1},<br/>{'name': 'Venusaur', 'height_m': 2.1}, {'name': 'Mega Venusaur', 'height_m': 2.4},<br/>{'name': 'Charmander', 'height_m': 0.6}, {'name': 'Charmeleon', 'height_m': 1.1},<br/>{'name': 'Charizard', 'height_m': 1.7}, {'name': 'Mega Charizard X', 'height_m': 1.7},<br/>{'name': 'Mega Charizard Y', 'height_m': 1.7}, {'name': 'Squirtle', 'height_m': 0.5},<br/>{'name': 'Wartortle', 'height_m': 1.1}]</code> |

Figure 5: This query returns a list of dictionaries with the `name` and `height_m` fields, for each dictionary in the `pokedex` parameter. The field `weight_kg` does not exist in the abridged `pokedex` and is ignored.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <b>Query:</b> <code>select</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |  |
| <pre>[{'pokedex_number': 1, 'name': 'Bulbasaur', 'type_2': 'Poison', 'height_m': 0.7}, {'pokedex_number': 2, 'name': 'Ivysaur', 'type_2': 'Poison', 'height_m': 1.1}, {'pokedex_number': 3, 'name': 'Venusaur', 'type_2': 'Poison', 'height_m': 2.1}, {'pokedex_number': 3, 'name': 'Mega Venusaur', 'type_2': 'Poison', 'height_m': 2.4}, {'pokedex_number': 4, 'name': 'Charmander', 'type_2': None, 'height_m': 0.6}, {'pokedex_number': 5, 'name': 'Charmeleon', 'type_2': None, 'height_m': 1.1}, {'pokedex_number': 6, 'name': 'Charizard', 'type_2': 'Flying', 'height_m': 1.7}, {'pokedex_number': 6, 'name': 'Mega Charizard X', 'type_2': 'Dragon', 'height_m': 1.7}, {'pokedex_number': 6, 'name': 'Mega Charizard Y', 'type_2': 'Flying', 'height_m': 1.7}, {'pokedex_number': 7, 'name': 'Squirtle', 'type_2': None, 'height_m': 0.5}, {'pokedex_number': 8, 'name': 'Wartortle', 'type_2': None, 'height_m': 1.1}]</pre> |  |

Figure 6: This query does not specify any fields. It returns a list of dictionaries with all fields.

- We will always test your code with well-formed statements.
- The keyword `select` is never used for any other purpose.
- Assume there are no spaces in field names.
- The field name in the query must exactly match the field name in the database. It is case sensitive.
- If a requested field is not present in the dataset, ignore it. See Figure 5.
- A PQL `select` query with no fields should return all fields. See Figure 6.
- Do not include empty dictionaries in the result.

#### Notes:

- **Do not alter the data referenced by the parameter** `pokedex`.
- The order of the dictionaries in the output list should be the same as the input list.
- The order of the fields in the dictionaries does not matter.

Use `q3b_test.py` to test your code. Your `q3b.py` should contain the function definition of `select_fields`. We will only evaluate your `q3b.py` with a different test script.

### Question 4 (Difficulty Level: \*\*\*)

[ 4 marks ]

In `q4.py`, implement a function called `calculate_actual_delays()`. This function takes in the following parameters:

- `list_cars` (type: list): a list of sub-lists of cars. Each sub-list represents the sequence of cars waiting for traffic lights on a given lane. In particular, the first sub-list contains all the cars in the 1<sup>st</sup> lane, the second sub-list contains all the cars in the 2<sup>nd</sup> lane, and so on. Each car is represented by a tuple `(car_id, delay)` in which `car_id` is a positive integer which is unique in `list_cars`, and `delay` is also a positive integer representing the time it takes (in seconds) for this car to clear the traffic light when there is no car in front of it.
- `traffic_lights` (type: tuple): a tuple of two integers `(green, red)`, in which `green` and `red` are the durations in seconds that the respective traffic light is on. Note that the green light duration is always at least as long as the longest `delay` value in `list_cars` so that even the slowest car will have enough time to clear the traffic light. There is no such restriction for red light durations.

The function returns a tuple `(list_cars_with_actual_delays, count)`, in which `list_cars_with_actual_delays` is a list of sub-lists of cars in their respective lanes as in `list_cars`,

except that the `delay` values for each car is **changed** to the **actual time duration** (in seconds) that this car has spent to clear the traffic light. The order of all elements in the list must be the same as that in `list_cars`. The `count` is the required number of green lights so that all cars can clear the traffic light. Assume that we always start with a green light, so `count` is at least 1. Your function **must not** modify `list_cars`.

To clear the traffic light, a car must wait until it becomes the first car of its lane, i.e., all the cars, if any, in front of it have already been cleared; and its `delay` must be **shorter or the same** as the remaining duration of the current green light. If the remaining duration of the green light is not enough, all remaining cars must wait for the remaining duration of the green light **and** the red light's duration before they can start clearing the traffic light again. Note that cars in different lanes can clear the traffic light in parallel.

Examples:

- `calculate_actual_delays([(1, 2), (2, 3), (3, 2), (4, 3)], (10, 5))` returns `([(1, 2), (2, 5), (3, 7), (4, 10)], 1)` in this exact order. There is only one lane with 4 cars in this case, and the green/red light durations are 10 seconds and 5 seconds, respectively. We note that car 2 must wait for car 1 to clear first, so the actual delay for car 2 would be the sum of the delay of car 1 and car 2's own delay, and so on. All the cars can clear the traffic light in only one green light duration, hence `count` is 1.
- `calculate_actual_delays([(1, 5), (2, 5), (3, 5), (4, 4)], (10, 5))` returns `([(1, 5), (2, 10), (3, 20), (4, 24)], 2)`. Note that all cars can clear the traffic light using two green light durations, hence `count` is 2. Car 3 must wait for car 1, car 2, and the second green light which only appears after a duration of 5 seconds for the red light, hence its actual delay is 20 seconds.
- `calculate_actual_delays([(1, 2), (2, 2), (3, 8)], [(4, 4), (5, 6), (6, 15), (7, 1)], [(8, 15), (9, 1)], (15, 10))` returns `([(1, 2), (2, 4), (3, 12)], [(4, 4), (5, 10), (6, 40), (7, 51)], [(8, 15), (9, 26)], 3)`. Note that car 6 can clear the traffic light in a total of 40 seconds. This is because it cannot clear the traffic light during the 1<sup>st</sup> green light duration (15 seconds). In this case, car 6 must wait for the remaining duration of the 1<sup>st</sup> green light. It then waits 10 seconds for the red light, and it finally takes another 15 seconds to clear the 2<sup>nd</sup> green light. Note that `count` is 3 because cars in the second lane need 3 green light durations to clear the traffic light.

Use `q4_test.py` to test your code. Your `q4.py` should only contain the function definition of `calculate_actual_delays`. We will only grade your `q4.py` using a different test script.

**END OF PAPER**